

NFC - Near Field Communication

Patrick Furrer

Simon Guggisberg

Jonas Troeltsch

Introduction

NFC est un ensemble de protocoles de communication sans fil qui permettent l'échange de données et le chargement de batterie et ce jusqu'à ~4cm. Se basant sur la technologie RFID (radio frequency identification), l'histoire de NFC remonte jusqu'en 1983. Ce n'est qu'en 2003 que la norme ISO NFC a été approuvée, et en 2006 que les spécifications pour NFC Tags ont été définies.

NFC Tag

Il s'agit d'un format de donnée écrit sur le tag : NDEF (NFC Data Exchange Format). Ce standard est défini par le NFC Forum.

- **Simple** : sémantique Read/Write (ou Read-Only)
- **Avancé** : opérations mathématiques, hardware cryptographique, environnement permettant exécution de code sur le tag

Android

Afin de pouvoir utiliser le lecteur NFC d'un téléphone Android dans une application, il faut demander la permission dans le manifest pour utiliser la technologie.

```
1 <uses-permission android:name="android.permission.NFC" />
```

xml

Tous les téléphones ne disposent pas d'un lecteur NFC. Pour que l'application ne s'affiche sur Google Play Store que pour les téléphones disposant d'un lecteur NFC, il faut ajouter ceci dans le manifest :

```
1 <uses-feature android:name="android.hardware.nfc" android:required="true" />
```

xml

Il est également recommandé de requérir une certaine version d'API car les fonctionnalités NFC ont été améliorées au fil des versions d'Android :

- 9+ : ACTION_NDEF_DISCOVERED et EXTRA_NDEF_MESSAGES
- 10+ : Bien meilleur support pour Reader/Writer et Foreground Dispatch System
- 14+ : Méthodes additionnelles pour créer des records NDEF

```
1 <uses-sdk android:minSdkVersion="10" />
```

xml

Deux modes distincts existent pour pouvoir interagir avec la technologie NFC du point de vue Android :

- **Reader/Writer mode** : pour lire et écrire des tags NFC
- **Émulation de carte** : pour émuler une carte NFC via un téléphone, qui peut ensuite être accédé par un lecteur de carte NFC externe.

Pour écrire dans un NFC tag, il faut établir son propre protocole de communication.

Données NDEF

Les données sont encapsulées dans un message NdefMessage qui contient un ou plusieurs enregistrements NdefRecord. Chaque enregistrement doit correspondre aux spécifications du type de données choisi.

Tag Dispatch System

Lorsque le téléphone est déverrouillé, et que l'option NFC est activée, le téléphone va scanner les tags NFC à proximité.

En cas de détection de tag, le TAG Dispatch System va :

1. Parser le NFC tag pour trouver le MIME type ou une URI qui identifie le payload de données dans le tag
2. Encapsuler le MIME type ou l'URI ainsi que le payload dans un intent
3. Démarrer une activité basée sur l'intent

Cet intent est pour éviter à l'utilisateur, une fois proche du NFC tag, de devoir choisir une application pour lire les données. En effectuant ce type de mouvement, la connexion pourrait être perdue, en raison de la très courte distance de connexion (~4cm).

Il est possible de déclarer un intent filter pour intercepter les intents et traiter les données.

Types d'actions

- ACTION_NDEF_DISCOVERED : pour lire des données NDEF (MIME type ou URI)
- ACTION_TECH_DISCOVERED : pour lire technologies NFC (NfcA, NfcB, NfcF ...)
- ACTION_TAG_DISCOVERED : si aucune des deux actions précédentes n'est définie

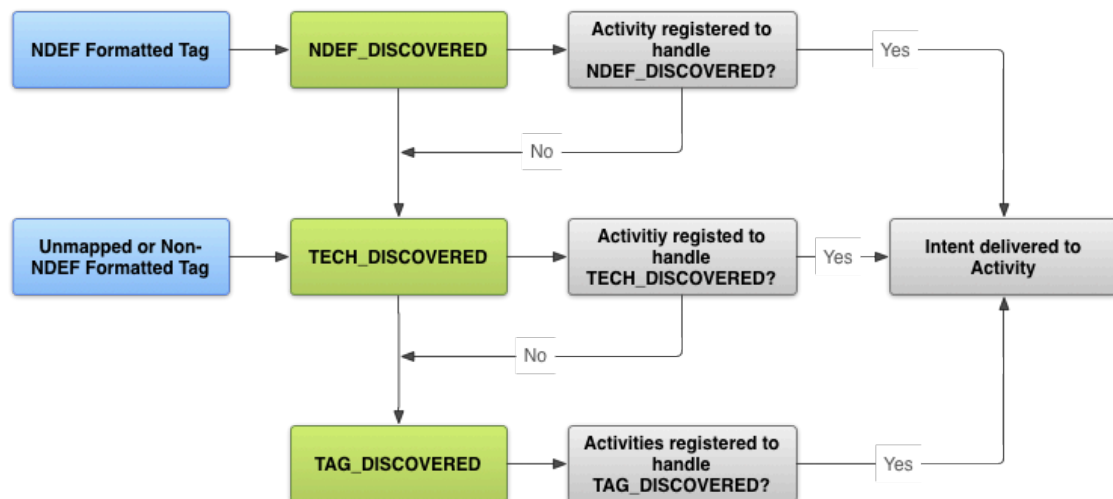


Fig. 1. – NFC Tag Dispatch System

Exemple NDEF

Exemple de déclaration d'intent, dans le manifest, pour le type text/plain :

```
1 <intent-filter>
2     <action android:name="android.nfc.action.NDEF_DISCOVERED" />
3     <category android:name="android.intent.category.DEFAULT" />
4     <data android:mimeType="text/plain" />
5 </intent-filter>
```

xml

Exemple de déclaration d'intent, pour une URI :

```
1 <intent-filter>
2     <action android:name="android.nfc.action.NDEF_DISCOVERED" />
3     <category android:name="android.intent.category.DEFAULT" />
4     <data android:scheme="https" android:host="developer.android.com"
5         android:pathPrefix="/index.html" />
6 </intent-filter>
```

xml

Exemple Technologies

Pour déclarer une intent au niveau des technologies, il faut d'abord créer une ressource les listant, sous project-root/res/xml/tech_list.xml.

Exemple de fichier de ressource acceptant technologies NfcA ET Ndef OU NfcB ET Ndef :

```
1 <resources xmlns:xliff="urn:oasis:names:tc:xliff:document:1.2">
2     <tech-list>
3         <tech>android.nfc.tech.NfcA</tech>
4         <tech>android.nfc.tech.Ndef</tech>
5     </tech-list>
6     <tech-list>
7         <tech>android.nfc.tech.NfcB</tech>
8         <tech>android.nfc.tech.Ndef</tech>
9     </tech-list>
10 </resources>
```

xml

Puis, dans le manifest, il faut ajouter un lien vers la ressource créée :

```
1 <activity>
2     ...
3     <intent-filter>
4         <action android:name="android.nfc.action.TECH_DISCOVERED" />
5     </intent-filter>
6
7     <meta-data android:name="android.nfc.action.TECH_DISCOVERED"
8         android:resource="@xml/nfc_tech_filter" />
9     ...
10 </activity>
```

xml

Exemple Tag Discovered

En cas de fallback sur l'action ACTION_TAG_DISCOVERED, cela se fait ainsi dans le manifest :

```
1 <intent-filter>
2     <action android:name="android.nfc.action.TAG_DISCOVERED" />
3 </intent-filter>
```

xml

Foreground Dispatch System

Foreground Dispatch System permet de bypasser le Tag Dispatch System pour lire des tags NFC qui auraient ouverts une autre activité, par l'activité courante, en premier plan.

Ceci se fait au travers de méthodes de la classe NfcAdapter qui doivent être activées et désactivées lorsque l'application est mise en pause, ou résumée.

À noter que, si l'on ne connaît ni les MIME type ni les technologies à filtrer, il est recommandé de les laisser à null lors de l'appel de enableForegroundDispatch.

```
1 private lateinit var adapter: NfcAdapter
2 private lateinit var pendingIntent: PendingIntent
3 private lateinit var intentFiltersArray: Array<IntentFilter>
4 private lateinit var techListsArray: Array<Array<String>>
5
6 override fun onResume() {
7     super.onResume()
8     adapter.enableForegroundDispatch(this, pendingIntent, intentFiltersArray,
9         techListsArray)
10 }
11
12 override fun onPause() {
13     super.onPause()
14     adapter.disableForegroundDispatch(this)
15 }
```

kotlin

Exemple de paramétrage de ces filtres. :

```
1 override fun onCreate(savedInstanceState: Bundle?) {
2     super.onCreate(savedInstanceState)
3     val intent = Intent(this, javaClass).apply {
4         addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP)
5     }
6     pendingIntent = PendingIntent.getActivity(
7         this, 0, intent,
8         PendingIntent.FLAG_MUTABLE
9     )
10    val ndef = IntentFilter(NfcAdapter.ACTION_NDEF_DISCOVERED).apply {
11        try {
12            addDataType("*/*")
13        } catch (e: IntentFilter.MalformedMimeTypeException) {
14            throw RuntimeException("fail", e)
15        }
16    }
```

kotlin

```
15     }
16 }
17
18     intentFiltersArray = arrayOf(ndef)
19     techListsArray = arrayOf(arrayOf(NfcA::class.java.name,
18                                     NfcV::class.java.name))
20     adapter = NfcAdapter.getDefaultAdapter(this)
21
22     setContentView(R.layout.activity_main)
23 }
```

Émulation de carte - Host Card Emulation - HCE

Cela peut se faire :

- par un élément sécurisé, une puce séparée, souvent hébergé sur une carte SIM, demande un matériel dédié.
- Entièrement côté software. Utilise les sécurités Android telles que Application Sandbox pour s'assurer que les données ne soient pas accessibles par d'autres applications.

Les services HCE sont basés sur Service de Android, ce qui les autorise à tourner en background sans interface utilisateur requise.

Pour pouvoir créer un Service HCE, il faut hériter de HostApuService et implémenter les méthodes processCommandApu et onDeactivated.

À noter qu'un Application Protocol Data Unit (ADPU) sont des paquets échangés entre un NFC reader et un HCE service.

```
1  class MyHostApuService : HostApuService() {
2
3      override fun processCommandApu(commandApu: ByteArray, extras: Bundle?):
4          ByteArray {
5          //called whenever a NFC reader sends an APDU
6          //TODO
7      }
8
9      override fun onDeactivated(reason: Int) {
10         //The link between the NFC reader and the device is broken
11         //TODO
12     }
13 }
```

Il faut ensuite modifier le manifest pour pointer, ici, vers un fichier apduservice.xml :

```
1  <service android:name=".MyHostApuService" android:exported="true"
2          android:permission="android.permission.BIND_NFC_SERVICE">
3      <intent-filter>
4          <action
5              android:name="android.nfc.cardemulation.action.HOST_APDU_SERVICE"/>
6      </intent-filter>
7      <meta-data android:name="android.nfc.cardemulation.host_apdu_service"
```

```
7         android:resource="@xml/apduservice"/>
8     </service>
```

Puis, dans le fichier `apduservice.xml` il faut déclarer les groupes d'Application Identification (AID).

Une AID est une chaîne de caractères unique qui permet d'identifier le type de service que l'application donne au lecteur NFC. Il est également possible de spécifier quelle application a la priorité, lorsqu'elle est en premier plan, pour quelle AID, ce qui peut être utile en cas de conflit si plusieurs services souhaitent traiter la même AID.

Ici, le groupe AID déclaré contient deux AIDs, plus d'informations ci-dessous.

```
1 <host-apdu-service xmlns:android="http://schemas.android.com/apk/res/
  android"
2     android:description="@string/servicedesc"
3     android:requireDeviceUnlock="false">
4     <aid-group android:description="@string/aiddescription"
5         android:category="other">
6         <aid-filter android:name="F0010203040506"/>
7         <aid-filter android:name="F0394148148100"/>
8     </aid-group>
9 </host-apdu-service>
```

Liens utiles

- NFC Forum : <https://nfc-forum.org/>
- Documentation Android NFC : <https://developer.android.com/develop/connectivity/nfc>